

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕНА К ЗАЩИТЕ

Заведующий кафедрой

_____ / Е.А. Пухова/

Руководитель образовательной
программы

_____ / М.В. Даньшина /

**«Разработка интеллектуальной веб-платформы управления бытовыми
задачами с контекстными советами и аналитикой привычек»**

Выпускная квалификационная работа

по направлению 09.03.01 Информатика и вычислительная техника

Образовательная программа (профиль) «Системная и программная
инженерия»

Студент: _____ / Рябкова Анна Сергеевна, 221-3210/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Кужалов Алексей Сергеевич, _____/
подпись *ФИО, уч. звание и степень*

Москва, 2026

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная инженерия»

ТЕМА ВКР	Разработка интеллектуальной веб-платформы управления бытовыми задачами с контекстными советами и аналитикой привычек
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	
Основные функции	
Используемые технологии и платформы	
ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	
Состав технической документации	
Состав графической части	

ПЛАН РАБОТЫ НАД ВКР

ЗАДАЧИ	НЕДЕЛИ																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Задача 1																		
Задача 2																		
Задача 3																		
Задача 4																		
...																		

РУКОВОДИТЕЛЬ ОП:

«__» _____ 2026, _____ / _____, _____ /
подпись ФИО, уч. звание и степень

РУКОВОДИТЕЛЬ ВКР:

«__» _____ 2026, _____ / _____, _____ /
подпись ФИО, уч. звание и степень

СТУДЕНТ:

«__» _____ 2026, _____ / _____, _____ /
подпись ФИО, группа

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ПРОЕКТНЫХ РЕШЕНИЙ	6
1.1 Анализ предметной области управления бытовыми задачами.....	6
1.2 Сравнительный анализ аналогов и конкурентов.....	11
1.3 Анализ программных инструментов разработки	14
1.4 Постановка задачи и требования к системе	20
1.5 Выводы по главе	31
2. ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ И ОПИСАНИЕ АЛГОРИТМОВ, ПРИМЕНЯЕМЫХ В РАБОТЕ	33
2.1 Алгоритм формирования контекстных рекомендаций на основе Retrieval-Augmented Generation	33
2.2 Алгоритм расчёта повторений и обработки пропусков.....	36
2.3 Проектирование пользовательского интерфейса	41
2.4 Техническая реализация программного продукта	46
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	50

ВВЕДЕНИЕ

Тема выпускной квалификационной работы — разработка интеллектуальной веб-платформы управления бытовыми задачами с контекстными советами и аналитикой привычек. Работа направлена на создание программного средства, предназначенного для планирования, распределения и контроля выполнения домашних обязанностей в условиях индивидуального и совместного использования.

Актуальность темы обусловлена тем, что в большинстве случаев бытовые задачи организуются с использованием разрозненных и неформализованных средств: устных договорённостей, бумажных списков, заметок и универсальных цифровых сервисов. Такой подход не обеспечивает прозрачности распределения обязанностей, затрудняет контроль регулярных и редких задач, не позволяет сохранять историю выполнения и проводить её анализ. В результате возрастает вероятность пропусков, неравномерного распределения нагрузки и повышения когнитивной нагрузки на участников бытового процесса.

Объектом исследования является процесс управления бытовыми задачами в условиях индивидуального и совместного быта. Предметом исследования являются методы и программные средства планирования, распределения, контроля выполнения домашних обязанностей, а также механизмы аналитической и контекстной поддержки пользователя.

Целью разработки является создание интеллектуальной веб-платформы управления бытовыми задачами для совместного и индивидуального использования, обеспечивающей планирование и контроль выполнения домашних дел, предоставление контекстных советов на основе базы знаний и аналитику привычек выполнения для решения проблемы мониторинга регулярности выполнения домашних дел и контроля редких задач.

Практическим результатом работы является программная реализация интеллектуальной веб-платформы, объединяющей средства создания и

назначения задач, ведения истории выполнения, расчёта аналитических показателей и формирования контекстных рекомендаций. Практическая значимость работы состоит в повышении прозрачности бытовых процессов, снижении риска пропуска задач, упрощении контроля регулярных обязанностей и создании единого цифрового пространства для распределения ответственности между участниками.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ПРОЕКТНЫХ РЕШЕНИЙ

1.1 Анализ предметной области управления бытовыми задачами

1.1.1 Термины и сущность

Управление бытовыми задачами представляет собой комплекс действий, направленных на поддержание порядка, функциональности и комфорта в доме. В контексте данной работы, бытовая задача определяется как дискретное действие или набор действий, необходимых для поддержания комфортных для жизни человека условий. Эти задачи могут быть регулярными (например, ежедневная уборка, приготовление пищи) или нерегулярными (например, сезонная чистка окон, ремонт бытовой техники).

Рутинa — это последовательность бытовых задач, выполняемых с определенной периодичностью, часто становящаяся автоматизированным поведением. Привычка в данном контексте рассматривается как устойчивая тенденция к выполнению определенных бытовых задач без сознательного усилия, формирующаяся через повторение и подкрепление. Формирование и поддержание полезных привычек является ключевым аспектом эффективного управления бытом [14].

Ответственность за выполнение бытовых задач может быть индивидуальной или совместной, когда несколько участников разделяют обязанности. Совместное планирование предполагает коллективное определение, распределение и координацию бытовых задач между участниками, что является критически важным для поддержания гармонии и эффективности в совместном быту [1].

1.1.2 Особенности регулярных и нерегулярных задач

Бытовые задачи характеризуются различной периодичностью и сложностью. Регулярные задачи требуют постоянного внимания и могут приводить к «выгоранию», если их выполнение не оптимизировано или не

распределено справедливо. Нерегулярные задачи легко забываются и накапливаются, создавая дополнительную эмоциональную нагрузку для человека [12, 13].

Одной из ключевых проблем является «пропуск» в расписании выполнения задач. Это может быть вызвано забывчивостью, отсутствием мотивации, недостатком времени или несправедливым распределением обязанностей. Пропуски приводят к нарушению рутины, накоплению невыполненных дел и, как следствие, к снижению общего уровня порядка и увеличению стресса у человека [12, 13].

1.1.3 Совместный быт как организационная система

Совместный быт можно рассматривать как сложную организационную систему, где каждый участник выполняет определенные функции. Эффективность такой системы зависит от нескольких факторов:

- **Распределение нагрузки:** Неравномерное распределение бытовых обязанностей является частой причиной конфликтов и недовольства [3]. Исследования показывают, что несправедливое распределение домашнего труда негативно сказывается на психологическом благополучии и удовлетворенности отношениями [1, 2, 3]. По данным ВЦИОМ, хотя 62% россиян выступают за равноправие в семье, включая распределение обязанностей, на практике сохраняется значительный гендерный разрыв в выполнении домашнего труда [2].
- **Прозрачность:** Отсутствие четкого понимания того, кто и что должен делать, а также статуса выполнения задач, приводит к дублированию усилий или, наоборот, к тому, что задачи остаются невыполненными.
- **Конфликтность:** Разногласия по поводу бытовых обязанностей являются одной из распространенных причин конфликтов в семьях и среди соседей по квартире. Цифровые инструменты могут помочь снизить конфликтность за счет объективизации распределения и контроля задач.

1.1.4 Влияние порядка в доме и когнитивной нагрузки бытовых задач на ментальное здоровье

Современные исследования подтверждают прямую связь между порядком в доме, эффективностью управления бытовыми задачами и ментальным благополучием человека. Беспорядок в жилом пространстве воспринимается мозгом как визуальный шум, что затрудняет концентрацию, вызывает хроническую усталость, апатию и может способствовать повышению уровня кортизола — гормона стресса. Организованное пространство, напротив, снижает психическую нагрузку и уровень стресса, улучшает качество сна и способствует психологическому благополучию.

Особое внимание уделяется концепции когнитивной нагрузки бытового труда (*cognitive household labor*) или ментальной нагрузки (*mental load*). Этот вид труда включает в себя планирование, предвосхищение потребностей, принятие решений и делегирование обязанностей, связанных с управлением домашним хозяйством. В отличие от физического выполнения задач, когнитивный труд часто является невидимым, безграничным и может выполняться фоном, параллельно с другими видами деятельности.

Исследования показывают, что когнитивная нагрузка бытового труда непропорционально ложится на женщин, что приводит к негативным последствиям для их психического здоровья, включая депрессию, стресс, выгорание и снижение удовлетворенности отношениями. Например, исследование Petts и Carlson (2023) выявило, что матери тратят в два раза больше времени на когнитивный труд, чем отцы, что коррелирует с повышенным риском стресса и депрессии. Аналогично, Ciciolla и Luthar (2021) установили, что чем больше задач мать воспринимает как зону её ответственности, тем ниже её благополучие и удовлетворенность отношениями.

Таким образом, неэффективное управление бытовыми задачами не только создает беспорядок, но и генерирует значительную когнитивную нагрузку, отвлекая человека от более важных задач и негативно влияя на его ментальное и

эмоциональное состояние. Это подчеркивает актуальность разработки системы, способной снизить эту нагрузку и оптимизировать бытовые процессы.

1.1.5 Текущее состояние процесса (AS-IS)

В настоящее время управление бытовыми задачами в большинстве домохозяйств осуществляется с использованием неформальных методов или простых инструментов. К ним относятся:

- Устная договоренность: задачи распределяются в процессе общения, что часто приводит к недопониманию и забывчивости.
- Бумажные списки/стикеры: простые списки задач, которые легко теряются или игнорируются.
- Общие календари (например, Google Calendar, Outlook Calendar): позволяют планировать события, но не имеют функциональности для отслеживания выполнения бытовых задач, распределения ответственности и аналитики.
- Специализированные приложения (например, FamilyWall, Nipto, Tody): частично решают проблему, но часто имеют ограничения в гибкости повторений, контекстных советах, аналитике привычек и адаптации к русскоязычному контексту.

Недостатки и информационно-когнитивные барьеры в текущем процессе (AS-IS):

1. Недостаточная прозрачность ответственности и статуса выполнения. При отсутствии единой системы фиксации задач усиливается субъективность оценок вклада и возрастает конфликтный потенциал при распределении домашних обязанностей.
2. Неравномерное распределение нагрузки между участниками. Эмпирические исследования и статистические наблюдения фиксируют устойчивые перекосы в распределении домашнего труда (в т.ч. гендерные),

что связано с восприятием несправедливости и ухудшением благополучия/ростом напряжённости в взаимоотношениях между участниками. [51]

3. Пропуски задач. При отсутствии формализованных правил переноса и обработки пропусков часть задач систематически забывается, а планирование теряет предсказуемость. С точки зрения поведенческой науки устойчивые практики требуют длительной повторяемости и подкрепления, а сбои снижают вероятность закрепления поведения. [50]
4. Отсутствие долговременной истории и аналитики выполнения. При разрозненных инструментах невозможно объективно оценить динамику исполнения, выявлять повторяющиеся паттерны и проводить управленческую коррекцию процесса. Для формирования привычек критична накопленная история повторений и измеримость прогресса, что подтверждается исследованиями в области. [50]
5. Дефицит процедурной и контекстной информации. Даже при наличии напоминания задача часто откладывается из-за неопределённости процесса и результата выполнения. В терминах поведенческой модели Фогга рост барьеров выполнения при отсутствии четко сформулированного задания и поддерживающей информации уменьшает вероятность совершения действия [15, 49].
6. Повышенная когнитивная нагрузка при планировании и принятии решений. Пользователю приходится самостоятельно оценивать приоритет, длительность, последовательность действий и точки начала выполнения; это повышает нагрузку на рабочую память и увеличивает вероятность откладывания и невыполнения. Теория когнитивной нагрузки показывает, что при высоких затратах когнитивных ресурсов эффективность выполнения и обучения действиям снижается [16, 48].
7. Недостаток информационной поддержки по реализации бытовых задач. Для повторяющихся, но вариативных задач наличие кратких чек-листов снижает вероятность пропусков шагов и ошибок за счёт функции внешней

когнитивной поддержки; эффективность чек-листов как инструмента предотвращения ошибок широко обоснована в исследованиях высоконагруженных доменов и переносима как принцип на бытовые процедуры [17, 18, 47]

1.2 Сравнительный анализ аналогов и конкурентов

1.2.1 Подход и критерии сравнения

Для сравнительного анализа существующих решений будет использован метод взвешенных критериев. Этот метод позволяет провести объективную оценку аналогов путем присвоения каждому критерию веса, отражающего его значимость для достижения целей проекта. Каждое решение будет оценено по шкале от 0 до 3, где:

- 0 – функциональность отсутствует;
- 1 – функциональность реализована минимально или с существенными ограничениями;
- 2 – функциональность реализована частично или не в полной мере;
- 3 – функциональность реализована полностью.

Итоговая оценка будет рассчитана как взвешенная сумма оценок по всем критериям. Критерии и их веса представлены в Таблице 1.

Таблица 1 – Критерии и веса для сравнения аналогов

Критерий	Вес	Обоснование
K1: Гибкость управления задачами	0.20	Возможность создавать регулярные и нерегулярные задачи с гибкими настройками повторения, а также корректная обработка пропусков.
K2: Функциональность совместного использования	0.20	Наличие ролей, возможность распределения задач, контроль выполнения и прозрачность для всех участников.

Критерий	Вес	Обоснование
К3: Аналитика и история	0.25	Хранение истории выполнения задач, формирование метрик привычек, визуализация данных для анализа.
К4: Контекстные советы и рекомендации	0.15	Наличие интеллектуальной составляющей предоставляющей пользователю релевантные советы по выполнению задач.
К5: Пользовательский интерфейс и опыт (UI/UX)	0.10	Интуитивно понятный интерфейс, простота использования, наличие мобильной версии или PWA
К6: Адаптация для русскоязычного рынка	0.10	Наличие русскоязычного интерфейса, адекватная ценовая политика и поддержка.

1.2.2 Классы решений и репрезентативные примеры

Существующие решения можно разделить на несколько классов:

1. Семейные органайзеры: приложения, ориентированные на семьи, с широким функционалом, включающим календари, списки покупок, заметки и управление задачами (например, FamilyWall) [35, 36].
2. Приложения для управления домашними делами (Chore Apps): специализированные приложения для распределения и отслеживания бытовых задач, часто с элементами геймификации (например, Nipto, Tody, ChoreBuster) [37, 38, 39, 40].
3. Таск-менеджеры общего назначения: приложения для управления задачами, которые могут быть адаптированы для бытовых нужд, но не имеют специализированного функционала (например, Todoist, Trello) [37, 38].
4. ИИ-помощники и платформы: решения, использующие искусственный интеллект для автоматизации и предоставления рекомендаций, но редко сфокусированные на бытовых задачах (например, Google Assistant, Amazon Alexa) [33, 34].

1.2.3 Сравнительная таблица аналогов

В Таблице 2 представлен сравнительный анализ репрезентативных аналогов по выбранным критериям.

Перед выставлением итоговых баллов каждый аналог был проанализирован по функциональности. Оценка по критерию выставлялась по шкале от 0 до 3 в зависимости от степени реализации соответствующей возможности: 0 — функциональность отсутствует, 1 — реализована минимально или с существенными ограничениями, 2 — реализована частично, 3 — реализована полноценно. При этом учитывалось не только формальное наличие функции, но и глубина её реализации применительно к задаче управления бытовыми делами. В частности, если система поддерживает назначение задач и напоминания, но не предоставляет гибких правил повторения и обработки пропусков, по критерию K1 выставлялась оценка 2, а не 3; если решение позволяет отслеживать участие членов семьи, но не содержит развитой аналитики и метрик привычек, по критерию K3 присваивалась оценка 1 или 2.

Таблица 2 – Сравнительная таблица аналогов

Решение	K1	K2	K3	K4	K5	K6	Итоговый балл
FamilyWall	2	3	1	0	2	1	1.5
Nipto	2	3	2	0	2	1	1.8
Tody	3	1	2	1	3	2	2.1
ChoreBuster	3	2	1	0	1	1	1.4
Todoist	3	2	1	0	3	3	1.8

Решение	K1	K2	K3	K4	K5	K6	Итоговый балл
Проектируемая система	3	3	3	3	3	3	3.0

1.2.4 Выводы по аналогам

Анализ существующих решений показывает, что ни одно из них не покрывает полностью все потребности пользователей в управлении бытовыми задачами. FamilyWall и Nipto хорошо справляются с совместным использованием, но имеют ограниченную аналитику и не предоставляют контекстных советов. Tody предлагает хорошую систему отслеживания чистоты, но слабее в совместном использовании. ChoreBuster автоматизирует распределение, но имеет устаревший интерфейс. Todoist гибок, но не специализирован для бытовых задач.

Разрабатываемая система нацелена занять нишу комплексного решения, объединяющего гибкое управление задачами, развитые функции совместного использования, глубокую аналитику привычек и интеллектуальные контекстные советы. Это позволит не только организовать бытовые процессы, но и формировать устойчивые полезные привычки, снижая когнитивную нагрузку и конфликтность в быту.

1.3 Анализ программных инструментов разработки

1.3.1 Веб-платформы и PWA

Для реализации интеллектуальной веб-платформы управления бытовыми задачами целесообразно использовать концепцию Progressive Web Applications (PWA). PWA представляют собой веб-приложения, которые благодаря современным веб-технологиям предоставляют функциональность, сравнимую с нативными мобильными приложениями [19, 46]. Основные преимущества PWA включают:

- **Доступность:** работают на любой платформе с современным браузером.
- **Установка:** могут быть установлены на домашний экран устройства без необходимости публикации в магазинах приложений.
- **Офлайн-режим:** благодаря Service Workers PWA могут кэшировать ресурсы и работать в условиях отсутствия или нестабильного интернет-соединения [22]. Service Workers перехватывают сетевые запросы и могут отдавать контент из кэша, обеспечивая бесперебойную работу [43].
- **Уведомления:** PWA могут отправлять push-уведомления пользователям, что критически важно для напоминаний о задачах, что реализуется через Push API и Notifications API [23, 25].
- **Синхронизация:** Background Synchronization API позволяет откладывать сетевые операции до восстановления стабильного соединения, обеспечивая надежную передачу данных даже при временных сбоях [46]. Это особенно важно для синхронизации статусов выполнения задач и аналитических данных.

Ограничения PWA связаны с доступом к некоторым нативным функциям устройства (например, глубокая интеграция с аппаратным обеспечением), однако для задач управления бытом текущие возможности PWA являются достаточными [47].

1.3.2 Архитектурные подходы

Выбор архитектурного подхода существенно влияет на масштабируемость, поддерживаемость и гибкость системы. Возможны следующие варианты:

- **Монолитная архитектура:** Все компоненты системы объединены в единое приложение. Простота разработки на начальных этапах, но сложность масштабирования и внесения изменений в дальнейшем [48].
- **Микросервисная архитектура:** Система разбита на небольшие, независимо разворачиваемые сервисы, взаимодействующие друг с другом.

Обеспечивает высокую масштабируемость и гибкость, но увеличивает сложность разработки и эксплуатации [49].

- Модульная архитектура: Компромисс между монолитом и микросервисами, где система разделена на слабосвязанные модули внутри одного приложения. Позволяет достичь хорошего баланса между гибкостью и управляемостью [50].

Для интеллектуальной веб-платформы, требующей обработки событий и аналитики привычек, целесообразно использовать модульную архитектуру с элементами Event Sourcing или Event Log [51]. Это означает, что все изменения состояния системы (например, выполнение задачи, изменение расписания) записываются как последовательность событий. Такой подход обеспечивает полную историю изменений, что является основой для глубокой аналитики и формирования контекстных советов [52].

1.3.3 Хранение данных и аналитика

Для хранения данных системы потребуется гибридный подход:

- Реляционная модель данных: для хранения основной информации о пользователях, домах, помещениях, задачах, расписаниях и ролях. PostgreSQL является оптимальным выбором благодаря своей надежности, широкой функциональности (включая поддержку JSONB для гибких схем) и возможностям для аналитики [27].
- Временные ряды/Агрегации: для отслеживания истории выполнения задач и формирования метрик привычек. PostgreSQL позволяет эффективно работать с временными рядами с использованием оконных функций и расширений [28]. Агрегированные данные (например, количество выполненных задач за период, среднее время выполнения) будут использоваться для расчета метрик привычек.

Метрики привычек будут включать:

- Коэффициент приверженности (Adherence Rate): процент выполненных задач от запланированных.
- Серия (Streak): количество последовательно выполненных задач.
- Коэффициент завершения (Completion Ratio): общий процент завершенных задач.
- Коэффициент просрочки (Overdue Rate): процент задач, выполненных после установленного срока.
- Распределение нагрузки: анализ объема и сложности задач, выполненных каждым участником домохозяйства.
- Вариативность (Variability): изменение паттернов выполнения задач. Оценка отклонения фактического времени выполнения от запланированного и разброс интервалов между повторяющимися выполнениями. Чем ниже вариативность, тем устойчивее сформирована привычка.

Эти метрики позволяют объективно оценивать формирование и поддержание привычек, а также выявлять проблемные зоны в управлении бытом.

1.3.4 Подходы к реализации контекстных рекомендаций

Реализация интеллектуальных контекстных советов может быть основана на различных подходах:

- База знаний (Rule-based systems): советы формируются на основе заранее определенных правил и условий. Простота реализации и предсказуемость, но ограниченная гибкость и сложность масштабирования.
- Машинное обучение (ML): использование алгоритмов для выявления паттернов в данных и формирования рекомендаций. Требует большого объема данных для обучения и может быть сложным в интерпретации.
- Большие языковые модели (LLM): применение генеративных моделей для создания естественных и контекстно-зависимых советов. Обеспечивает

высокую гибкость и качество советов, но сопряжено с рисками галлюцинаций (генерация недостоверной информации) [41, 45].

Для данной платформы целесообразно использовать гибридный подход: база знаний для типовых и критически важных советов (например, напоминания о регулярных задачах, базовые рекомендации по распределению нагрузки) в сочетании с ML/LLM для формирования более сложных и персонализированных контекстных советов. Контекстные советы будут учитывать текущее состояние (например, время суток, погода, наличие других задач), историю выполнения и предпочтения пользователя. Для минимизации рисков галлюцинаций LLM-модели будут использоваться со строгой модерацией и верификацией генерируемых советов [41, 45].

1.3.5 Обоснование выбора технологий

Поскольку разрабатываемая система является веб-платформой с PWA-функциями, целесообразно разделять технологический стек на клиентский слой (UI/PWA) и серверный слой (API, бизнес-логика, безопасность, доступ к данным). В рамках данной работы клиентский слой фиксируется как веб-клиент с поддержкой PWA (Service Worker, Web App Manifest, push-уведомления), а сравнительный анализ выполняется для серверного слоя на Python, так как именно он определяет: транзакционность, модель данных, реализацию расписаний/политик пропусков, ведение журнала событий и расчёт аналитических метрик [20, 21].

Для выбора оптимального технологического стека проведем сравнительный анализ трех популярных вариантов с использованием метода взвешенных критериев. Критерии и их веса представлены в Таблице 3.

Таблица 3 – Критерии для сравнения технологического стека

Критерий	Вес	Обоснование
K1: Производительность масштабируемость	0.25	Способность системы обрабатывать большое количество запросов и пользователей, а также легко масштабироваться.
K2: Скорость и простота разработки	0.20	Наличие готовых библиотек, фреймворков и инструментов для быстрой разработки.
K3: Интеграция с PWA архитектурой	0.20	Наличие инструментов и библиотек для реализации офлайн-режима, уведомлений, синхронизации.
K4: Экосистема и сообщество	0.15	Доступность документации, готовых решений, активное сообщество разработчиков.
K5: Стоимость владения развертывания	0.10	Затраты на лицензии, инфраструктуру, специалистов.
K6: Возможности для ИИ/ML	0.10	Интеграция с библиотеками для машинного обучения и анализа данных.

Таблица 4 – Сравнение вариантов технологического стека

Стек	K1	K2	K3	K4	K5	K6	Итоговый балл
Вариант 1: Python (FastAPI) + PostgreSQL React/Next.js	3	3	3	3	3	3	3.0
Вариант 2: Node.js (Express) + MongoDB Vue.js	2	3	3	3	3	2	2.6
Вариант 3: Java (Spring Boot) + MySQL Angular	3	2	2	2	2	2	2.3

Обоснование выбора.

По результатам сравнительного анализа, стек Python (FastAPI) + PostgreSQL + React/Next.js демонстрирует наилучшие показатели. FastAPI обеспечивает высокую производительность и асинхронность, что критично для интерактивной веб-платформы с интеллектуальными функциями [30]. PostgreSQL предоставляет надежное и гибкое хранилище данных, идеально подходящее для реляционных и временных рядов, а также для аналитики. React/Next.js является мощным фреймворком для создания современных PWA с

отличной поддержкой офлайн-режима и уведомлений. Экосистема Python также предлагает богатые возможности для интеграции ML/AI моделей, что является ключевым для реализации контекстных советов.

1.4 Постановка задачи и требования к системе

1.4.1 Формализация проблемы

Проблема управления бытовыми задачами формализуется как отсутствие эффективной системы для планирования, распределения, отслеживания выполнения и анализа домашних обязанностей в условиях совместного проживания. В терминах процессов, это приводит к неоптимальному процессу управления задачами, характеризующемуся низкой прозрачностью, неравномерным распределением нагрузки и отсутствием механизмов для формирования устойчивых привычек.

Данные, связанные с бытовыми задачами (тип задачи, периодичность, ответственный, статус выполнения), часто фрагментированы, несистематизированы и не агрегируются для анализа. События, такие как «задача выполнена», «задача просрочена», «задача назначена», не фиксируются централизованно, что исключает возможность построения полной истории и проведения аналитики.

Ограничения предметной области включают:

- Разнообразие бытовых задач: от простых до сложных, требующих разного подхода к планированию и выполнению.
- Человеческий фактор: забывчивость, отсутствие мотивации, нежелание выполнять рутинные задачи.
- Конфликт интересов: различные представления о справедливости распределения обязанностей между участниками домохозяйства.
- Отсутствие стандартизации: отсутствие единых правил и метрик для оценки эффективности управления бытом.

1.4.2 Цель и задачи разработки

Цель разработки: создать интеллектуальную веб-платформу управления бытовыми задачами для совместного и индивидуального использования, обеспечивающую планирование и контроль выполнения домашних дел, предоставление контекстных советов на основе базы знаний и аналитику привычек выполнения для решения проблемы мониторинга регулярности выполнения домашних дел и контроля редких задач.

Задачи разработки:

1. Разработать модуль гибкого планирования задач: реализовать функциональность создания регулярных и нерегулярных задач с настраиваемой периодичностью, обработкой пропусков и возможностью динамического перераспределения ответственности.
2. Создать систему ролей и распределения ответственности: обеспечить возможность назначения администраторов и участников дома, гибкое управление правами и прозрачное распределение задач между ними.
3. Внедрить подсистему аналитики и истории выполнения: разработать механизмы для сбора, хранения и визуализации данных о выполнении задач, расчёта метрик привычек и формирования отчётов по участникам/помещениям/задачам.
4. Разработать базу знаний бытовых задач и механизм её сопровождения: сформировать структуру знаний (карточки задач, чек-листы, необходимые ресурсы, рекомендуемая периодичность, условия применимости), определить формат хранения, а также реализовать инструменты администрирования и актуализации базы знаний (создание, редактирование, деактивация, связывание с типами задач и контекстами).
5. Реализовать модуль контекстных советов на основе базы знаний: разработать механизм подбора рекомендаций по выполнению задач на основании параметров задачи, контекста (помещение/зона, приоритет, доступное время) и истории выполнения.

6. Обеспечить кроссплатформенность и офлайн-доступ: разработать веб-платформу как PWA, поддерживающую работу в офлайн-режиме, push-уведомления и синхронизацию данных.

1.4.3 Функциональные требования

ФТ-1: Система должна предоставлять возможность создания и редактирования бытовых задач с указанием названия, описания, привязки к помещению/зоне, периодичности (разовая/ежедневно/еженедельно/ ежемесячно/ пользовательский интервал), плановой даты/времени или окна выполнения, срока и ответственного.

ФТ-2: Система должна поддерживать политики обработки пропусков для регулярных задач: (а) перенос ближайшего невыполненного экземпляра на ближайшее допустимое окно; (б) фиксация просрочки без переноса; (в) пропуск экземпляра без образования долга. Выбор политики задаётся при настройке задачи.

ФТ-3: Система должна обеспечивать создание «домов» и приглашение в них участников с различными ролями (Администратор, Участник, Ограниченный участник).

ФТ-4: Система должна позволять Администратору управлять правами доступа участников к задачам и настройкам дома.

ФТ-5: Система должна предоставлять интерфейс для отметки статусов задач участниками (выполнено/пропущено/перенесено) с фиксацией времени и автора действия.

ФТ-6: Система должна вести историю событий по задачам (создание, изменение, назначение, выполнение, перенос, пропуск) с фиксацией даты, времени и исполнителя/инициатора.

ФТ-7: Система должна рассчитывать и отображать метрики привычек для выбранного периода и уровня агрегации (участник/дом/помещение/задача): adherence rate, streak, completion ratio, overdue rate, распределение нагрузки.

ФТ-8: Система должна предоставлять контекстные советы по выполнению задач на основе базы знаний и параметров задачи (зона, периодичность, приоритет) и данных истории выполнения (например, просрочки/серии).

ФТ-9: Система должна поддерживать отправку push-уведомлений о предстоящих задачах, просроченных задачах и назначениях задач (при наличии согласия пользователя).

ФТ-10: Система должна обеспечивать работу в офлайн-режиме с сохранением действий пользователя в локальной очереди событий и последующей синхронизацией при восстановлении соединения.

ФТ-11: Система должна предоставлять возможность создания и управления помещениями/зонами внутри дома, к которым могут быть привязаны задачи.

ФТ-12: Система должна позволять участникам добавлять задачи другим участникам в рамках своих прав с фиксацией автора назначения.

ФТ-13: Система должна обеспечивать ограничения для роли «Ограниченный участник»: доступ к просмотру и отметке выполнения задач только в назначенных помещениях/зонах; запрет на изменение настроек дома и правил повторения.

ФТ-14: Система должна предоставлять Администратору средства управления базой знаний советов (создание/редактирование/деактивация рекомендаций, задание условий применимости к типам задач/зонам).

1.4.4 Нефункциональные требования

Нефункциональные требования (НФТ) определяют, насколько хорошо система выполняет свои функции. Для классификации НФТ используется модель качества программных продуктов ISO/IEC 25010 [5, 6].

НФТ-1 (Функциональная пригодность). Система должна корректно реализовывать функциональные требования; критерий приемки — прохождение не менее 95% тест-кейсов без критических дефектов и 100% критических

сценариев (создание дома, приглашение, создание задачи, отметка выполнения, синхронизация, просмотр отчёта).

НФТ-2 (Производительность). Для операций: (а) получение списка задач «на сегодня», (б) отметка выполнения задачи, (в) получение отчёта по метрикам за период 30 дней — время ответа серверного API должно составлять $p95 \leq 2,0$ с при 100 одновременных активных пользователях.

НФТ-3 (Производительность: расчёт аналитики). Расчёт агрегированных метрик (коэффициент завершения, коэффициент приверженности, коэффициент просрочки, серия, распределение нагрузки) для дома до 20 участников за период 90 дней должен выполняться за ≤ 5 с на сервере.

НФТ-4 (Совместимость). Клиентская часть должна корректно работать в последних двух стабильных версиях браузеров Chrome, Firefox, Edge и в актуальной версии Safari (настольной и мобильной) для набора сценариев: вход, просмотр задач, создание задачи, отметка выполнения, просмотр отчёта, офлайн-режим.

НФТ-5 (Доступность). В режиме опытной эксплуатации сервис должен обеспечивать доступность не ниже 99,5% в месяц, исключая плановые окна обновления; критерий — журнал мониторинга/логов доступности.

НФТ-6 (Надёжность данных). При переходе в офлайн-режим действия пользователя должны сохраняться локально и отправляться на сервер после восстановления сети; критерий — успешная синхронизация $\geq 99\%$ событий в тестах отключения сети без потери данных.

НФТ-7 (Безопасность). Пароли (при локальной аутентификации) должны храниться в виде стойких хешей; передача данных должна осуществляться только по HTTPS; критерий — проверка конфигурации и отсутствие хранения паролей в открытом виде в БД/логах.

НФТ-8 (Безопасность). Система должна обеспечивать разграничение доступа на основе ролей (RBAC) и контекстов (помещения/зоны для ограниченного участника); критерий — набор негативных тестов,

подтверждающий невозможность доступа к запрещённым операциям/данным [9].

НФТ-9 (Безопасность). Система должна быть защищена от типовых веб-угроз (XSS, CSRF, SQL-инъекции) средствами фреймворка/валидации ввода/параметризованных запросов; критерий — прохождение базового профиля сканирования без уязвимостей уровня High [10].

НФТ-10 (Удобство использования). Ключевые сценарии (создать задачу, отметить выполнение, просмотреть отчёт) должны выполняться пользователем за ≤ 5 действий (кликов/тапов) каждый; критерий — проверка по протоколу UX-оценки на макете/прототипе.

НФТ-11 (Сопровождаемость). Код должен иметь модульную структуру (разделение слоёв: API/домен/доступ к данным), документированные публичные интерфейсы и тесты доменной логики расписаний/пропусков; критерий — наличие README/архитектурного описания и покрытие unit-тестами ключевых модулей не ниже 60% (или иного значения, закреплённого в плане испытаний).

НФТ-12 (Переносимость). Развёртывание должно быть воспроизводимым с использованием контейнеризации; критерий — успешный запуск системы на чистом окружении по инструкции (docker compose) без ручных правок конфигурации.

1.4.5 Требования к условиям эксплуатации и техническим средствам

Клиентская часть.

Веб-платформа должна быть доступна через современные веб-браузеры на настольных компьютерах и мобильных устройствах: Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari. Поддержка функций PWA должна обеспечиваться в пределах возможностей целевых браузеров и операционных систем (Android/iOS).

Серверная часть.

Серверное приложение должно развёртываться в контейнеризованном окружении (Docker) на облачной или локальной инфраструктуре. Допускаются

платформы уровня IaaS/PaaS (например, виртуальная машина или контейнерный сервис). Минимальные ресурсы для стенда ВКР: 2 vCPU, 4 GB RAM, SSD-хранилище.

База данных.

В качестве СУБД используется PostgreSQL версии 14 или выше. Должны быть предусмотрены резервное копирование и механизмы миграций схемы данных, обеспечивающие воспроизводимость обновлений [29].

Сетевая безопасность и защита данных.

Весь трафик между клиентом и сервером должен передаваться по HTTPS. Аутентификация пользователей должна поддерживать локальную аутентификацию (логин/пароль) с хранением паролей в виде стойкого хеша и/или внешнюю аутентификацию через провайдера идентификации по протоколам OAuth 2.0 / OpenID Connect (при наличии выбранного провайдера).

Система должна обеспечивать базовые меры защиты веб-приложения: валидацию входных данных, параметризованные запросы к БД, защиту от XSS/CSRF, корректные настройки CORS и безопасное хранение токенов/сессий.

1.4.6 Концептуальная модель данных

Концептуальная модель данных описывает основные сущности системы и связи между ними, представлена на Рисунке 1 [11].

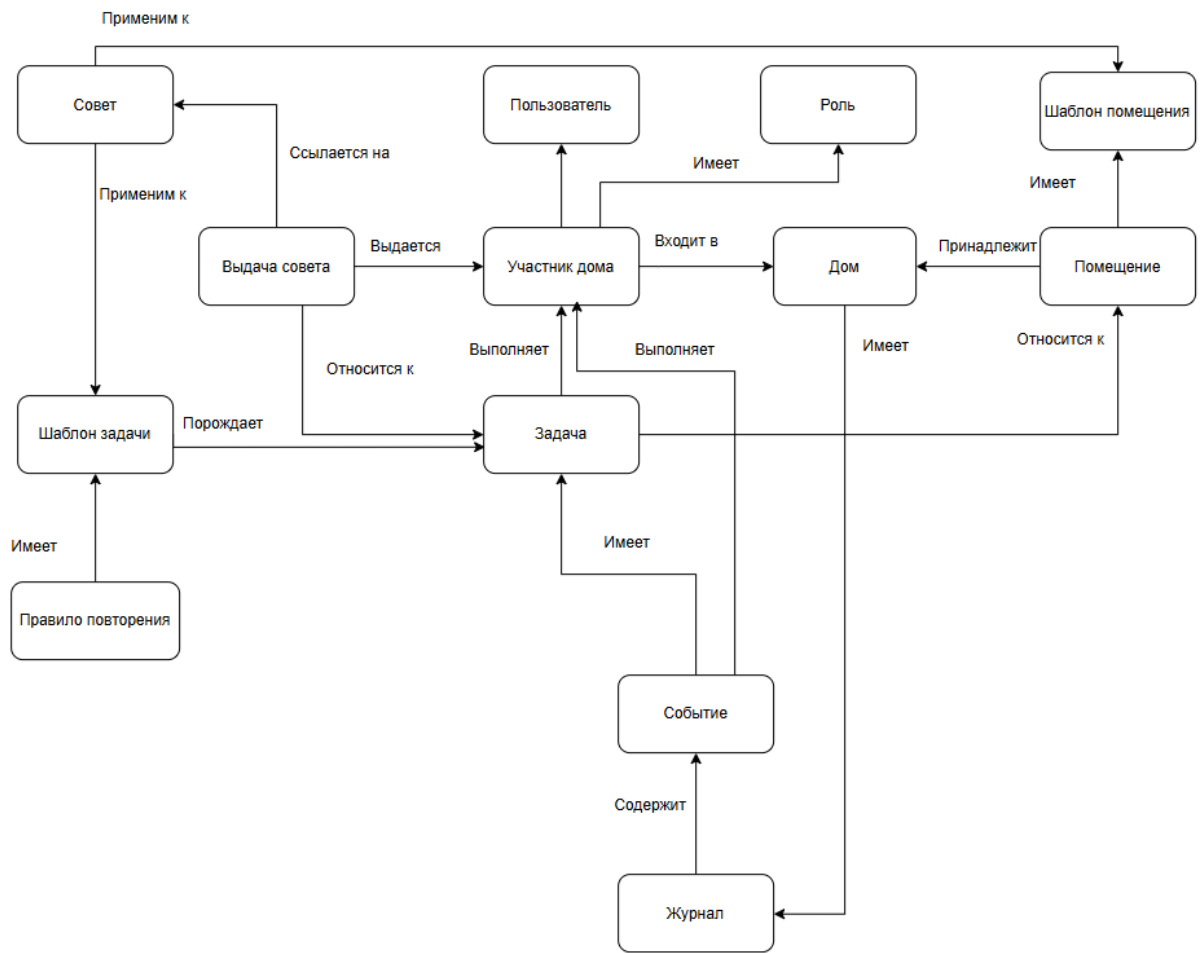


Рисунок 1 – Концептуальная модель данных

Основные сущности:

- **Пользователь:** хранит сведения о зарегистрированной учетной записи и выступает глобальной сущностью, не привязанной к конкретному дому. Связь пользователя с конкретным домом реализуется через сущность участника дома.
- **Участник дома:** представляет членство пользователя в конкретном доме и задает контекст его взаимодействия с системой. Данная сущность связывает пользователя с домом, определяет назначенную роль, используется для фиксации участия в выполнении задач, получения выдач советов и инициирования событий в журнале.

- Роль: описывает набор прав и возможностей участника дома в рамках конкретного дома (Администратор, Участник дома, Ограниченный участник).
- Дом: представляет виртуальное пространство совместного управления бытовыми задачами. Дом включает помещения, содержит задачи и связан с журналом, в котором аккумулируются события, формирующие историю выполнения.
- Шаблон помещения: задает типовой состав и структуру помещений и применяется для порождения конкретных помещений в доме, например при первичной настройке пространства.
- Помещение: описывает комнату или зону внутри дома и выступает контекстом выполнения задач. Помещение используется для группирования задач и для ограничения доступа в сценариях, где права участника определяются назначенными зонами.
- Шаблон задачи: описывает типовую задачу или рутину, служащую основой для создания конкретных задач. К шаблону задачи привязываются правила повторения, а также область применимости советов, что обеспечивает согласованность планирования и информационной поддержки.
- Правило повторения: задает параметры регулярности выполнения для шаблона задачи, включая периодичность и интервалы. Данная сущность формирует основу расписания и влияет на появление задач во времени.
- Задача: является конкретной единицей планирования и исполнения в рамках дома. Задача порождается на основе шаблона задачи, относится к определенному помещению и участвует в формировании истории через связанные события, отражающие изменения состояния и факты выполнения.
- Событие: фиксирует значимое действие или изменение, связанное с задачей, включая выполнение, перенос, пропуск и иные операции. Событие инициируется участником дома и используется как первичный источник данных для истории и аналитики.

- Журнал: агрегирует события, обеспечивая накопление и хранение истории действий в пределах дома. Журнал формирует основу для последующего анализа регулярности и поведения пользователей.
- Совет: является элементом базы знаний, предназначенным для информационной поддержки выполнения задач. Совет имеет область применимости, заданную через привязку к шаблонам задач и шаблонам помещений, и используется при формировании конкретных рекомендаций.
- Выдача совета: фиксирует факт предоставления совета участнику дома в контексте конкретной задачи. Данная сущность обеспечивает трассируемость рекомендаций, то есть возможность установить, какой совет был выдан, кому и к какой задаче он относился, что необходимо как для аудита, так и для последующей оценки эффективности рекомендаций.

1.4.7 Описание целевого состояния процесса

После внедрения интеллектуальной веб-платформы управления бытовыми задачами целевое состояние процесса (ТО-ВЕ) характеризуется следующими аспектами:

1. Единое пространство планирования и ответственности. Все задачи создаются и ведутся в рамках дома как общего пространства, при этом каждое действие имеет назначенного участника дома, определённую роль и правами. Назначение задач, изменение параметров и фиксация статусов происходят в системе, что устраняет неоднозначность и обеспечивает согласованное понимание текущего состояния работ всеми участниками.
2. Прозрачный контроль исполнения и фиксация статусов. Платформа предоставляет унифицированные статусы задач и механизмы отметки выполнения. Для каждого дома и участника формируется: перечень задач, сроки, состояние и история изменений. Наблюдаемость обеспечивается за

счёт регистрации событий, что снижает конфликтность, возникающую при субъективной оценке вклада.

3. Управляемое и устойчивое расписание с обработкой пропусков. Регулярные задачи формируются по правилам повторения, привязанным к шаблонам задач, а обработка пропусков осуществляется по заранее выбранной политике. Это обеспечивает предсказуемость планирования: система либо переносит пропущенный экземпляр в допустимое окно, либо фиксирует просрочку, либо закрывает пропуск без накопления «долга» в соответствии с выбранными настройками.
4. Снижение риска забывания за счёт автоматизации и уведомлений. План задач формируется автоматически на основе правил повторения и контекста дома, а участники получают уведомления о предстоящих и просроченных задачах. Такой механизм поддерживает регулярность исполнения и особенно важен для нерегулярных и редких задач, которые чаще всего теряются.
5. Долговременная история, измеримость и аналитика поведения. Все изменения состояния задач и факты выполнения фиксируются как события и агрегируются в журнале дома. На основе журнала рассчитываются метрики регулярности и поведения, позволяющие объективно оценивать выполнение задач, выявлять устойчивые отклонения и поддерживать управленческую коррекцию процесса. Это решает проблему отсутствия истории и превращает выполнение бытовых задач в измеримый процесс.
6. Контур справедливого распределения нагрузки. Платформа предоставляет отчёты о распределении выполнений и нагрузке по участникам, что позволяет выявлять перекосы и принимать решения о переназначении задач или изменении расписаний. В результате снижается вероятность систематической перегрузки отдельных участников и повышается воспринимаемая справедливость распределения обязанностей.
7. Контекстная информационная поддержка и снижение когнитивной нагрузки. Для типовых задач формируется база знаний в виде советов и

чек-листов, связанных с шаблонами задач и контекстом помещений. В момент взаимодействия с задачей система предоставляет пользователю структурированную инструкцию выполнения, перечень шагов и рекомендации, что снижает неопределённость, уменьшает когнитивные издержки планирования и повышает вероятность доведения задачи до завершения.

8. Трассируемость рекомендаций и контроль качества информационной поддержки. Факт предоставления рекомендаций фиксируется сущностью выдачи совета, связывающей совет, участника дома и конкретную задачу. Это обеспечивает проверяемость и аудит: можно установить, какие рекомендации были выданы, в каком контексте и кому, а также оценить их полезность через последующие события выполнения.

1.5 Выводы по главе

В первой главе был проведен анализ предметной области управления бытовыми задачами, обоснована актуальность разработки интеллектуальной веб-платформы и сформулированы ключевые требования к ней. Выявлено, что существующие подходы к организации домашних дел, включая специализированные приложения, не обеспечивают комплексного решения проблем, связанных с гибким планированием, справедливым распределением нагрузки, формированием устойчивых привычек и предоставлением контекстных советов. Это приводит к снижению эффективности бытовых процессов, росту когнитивной нагрузки и потенциальным конфликтам.

Сравнительный анализ аналогов показал, что проектируемая система способна занять уникальную нишу, предлагая интегрированный подход к управлению бытом с акцентом на аналитику привычек и интеллектуальную поддержку. Обоснован выбор технологического стека на базе Python (FastAPI), PostgreSQL и React/Next.js, который обеспечивает высокую производительность,

масштабируемость и широкие возможности для реализации PWA-функционала и интеграции ML/AI моделей.

Сформулированы функциональные и нефункциональные требования к системе, классифицированные в соответствии с моделью качества ISO/IEC 25010, а также разработана концептуальная модель данных. Представлены схемы текущего (AS-IS) и целевого (TO-BE) состояний процесса управления бытовыми задачами, демонстрирующие трансформационный потенциал разрабатываемой платформы. Полученные результаты формируют аналитическую базу для дальнейшего проектирования и разработки системы, направленной на повышение качества жизни пользователей за счет оптимизации бытовых процессов.

2. ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ И ОПИСАНИЕ АЛГОРИТМОВ, ПРИМЕНЯЕМЫХ В РАБОТЕ

В рамках разработки интеллектуальной веб-платформы управления бытовыми задачами применяются алгоритмы, обеспечивающие контекстную информационную поддержку пользователей и устойчивое планирование повторяющихся задач. Представлено описание двух алгоритмов: алгоритма формирования контекстных рекомендаций на основе Retrieval-Augmented Generation и алгоритма интеллектуального расчёта повторений с обработкой пропусков на базе формализации iCalendar.

2.1 Алгоритм формирования контекстных рекомендаций на основе Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) представляет собой подход, в котором генерация ответа языковой модели дополняется извлечением релевантных фрагментов из внешнего корпуса документов, выступающего в роли непараметрической памяти. В классической постановке RAG объединяет генеративную модель и механизм поиска по индексу документов, что повышает фактичность и специфичность выдачи за счёт опоры на извлечённые источники [41].

В рассматриваемой системе RAG применяется для формирования контекстных советов по выполнению бытовых задач. Рекомендация формируется с опорой на базу знаний и историю выполнения: сначала из корпуса инструкций, чек-листов и справочных материалов извлекаются наиболее релевантные фрагменты, после чего языковая модель формирует итоговый совет с учётом ограничений домена и контекста задачи. Обновление знаний достигается путём актуализации индексируемого корпуса без переобучения генеративной модели [41, 45].

2.1.1 Входные и выходные данные

Вход алгоритма включает описание задачи и контекста выполнения: идентификатор шаблона задачи, помещение, плановую дату, статус и приоритет, а также параметры участника дома и его настройки уведомлений. Дополнительно используется история событий выполнения за заданный период. Корпус знаний состоит из документов базы знаний, представленных в виде фрагментов текста с метаданными применимости. Выходом алгоритма является набор советов Тор-К и запись о факте выдачи совета для обеспечения трассируемости.

2.1.2 Этапы алгоритма

Алгоритм включает этап индексирования корпуса и этап онлайн выдачи. На этапе индексирования документы сегментируются на фрагменты, каждому фрагменту сопоставляется векторное представление, после чего фрагменты сохраняются в индекс для семантического поиска. На этапе онлайн выдачи формируется поисковый запрос, выполняется извлечение релевантных фрагментов, и далее генеративная модель получает эти фрагменты в качестве контекста для построения ответа.

Для семантического извлечения применим плотный модуль поиска, реализуемый в виде дуального энкодера – подход, описанный в работах по Dense Passage Retrieval [42]. Для хранения и поиска по векторным представлениям применимы библиотеки эффективного поиска ближайших соседей, например Faiss, предназначенная для поиска по сходству (Similarity search) и кластеризации плотных векторов [43].

Псевдокод алгоритма RAG-рекомендаций представлен на Листинге 1.

```
Алгоритм RAG_Recommend(Задача t, Участник u, История H, Индекс I, K):  
  q ← BuildQuery(t, u, H)      // формирование запроса из контекста  
  vq ← Embed(q)                // векторизация запроса  
  C ← RetrieveTopN(I, vq, N)    // извлечение N фрагментов знаний  
  P ← BuildPrompt(t, u, H, C)  // сборка контекста для генерации
```

```

r ← Generate(P)           // генерация совета языковой моделью
A ← PostProcess(r)        // нормализация: шаги, предупреждения,
краткость
SaveAdviceDelivery(u, t, A, C) // запись факта выдачи и
использованных фрагментов
return TopK(A)

```

Листинг 1 – Псевдокод алгоритма RAG-рекомендаций

Для обеспечения контролируемости и воспроизводимости применяется принцип ограниченной генерации: языковой модели передаются только релевантные фрагменты из базы знаний, а формат ответа нормализуется постобработкой. В сущности выдачи совета сохраняются как итоговая рекомендация, так и основание выдачи, включая перечень использованных фрагментов. Это обеспечивает проверяемость рекомендаций и снижает риск выхода за рамки предметной области, что соответствует концепции объединения параметрической и непараметрической памяти в RAG [41].

2.1.3 Структура базы знаний и организация корпуса рекомендаций

База знаний в разрабатываемой системе выступает источником непараметрической памяти для модуля контекстных рекомендаций. Ее структура ориентирована на поддержку релевантного поиска и последующего формирования советов с учетом предметного контекста бытовой задачи. Для этого элементы базы знаний организуются по нескольким классификационным измерениям: помещению, сезону и категории применения. Такой подход позволяет учитывать пространственный контекст выполнения задачи, временные условия применимости рекомендаций и тип бытовой активности.

Помимо тематической классификации, структура базы знаний включает контур индексирования и выдачи рекомендаций. Документы корпуса преобразуются в индексируемые элементы, используемые в процессе поиска релевантных фрагментов по пользовательскому запросу. Результат обращения

к базе знаний фиксируется в таблице выдачи совета, что обеспечивает отслеживаемость рекомендаций и возможность последующего анализа их применимости и полезности. Диаграмма структуры базы знаний представлена на Рисунке 2.

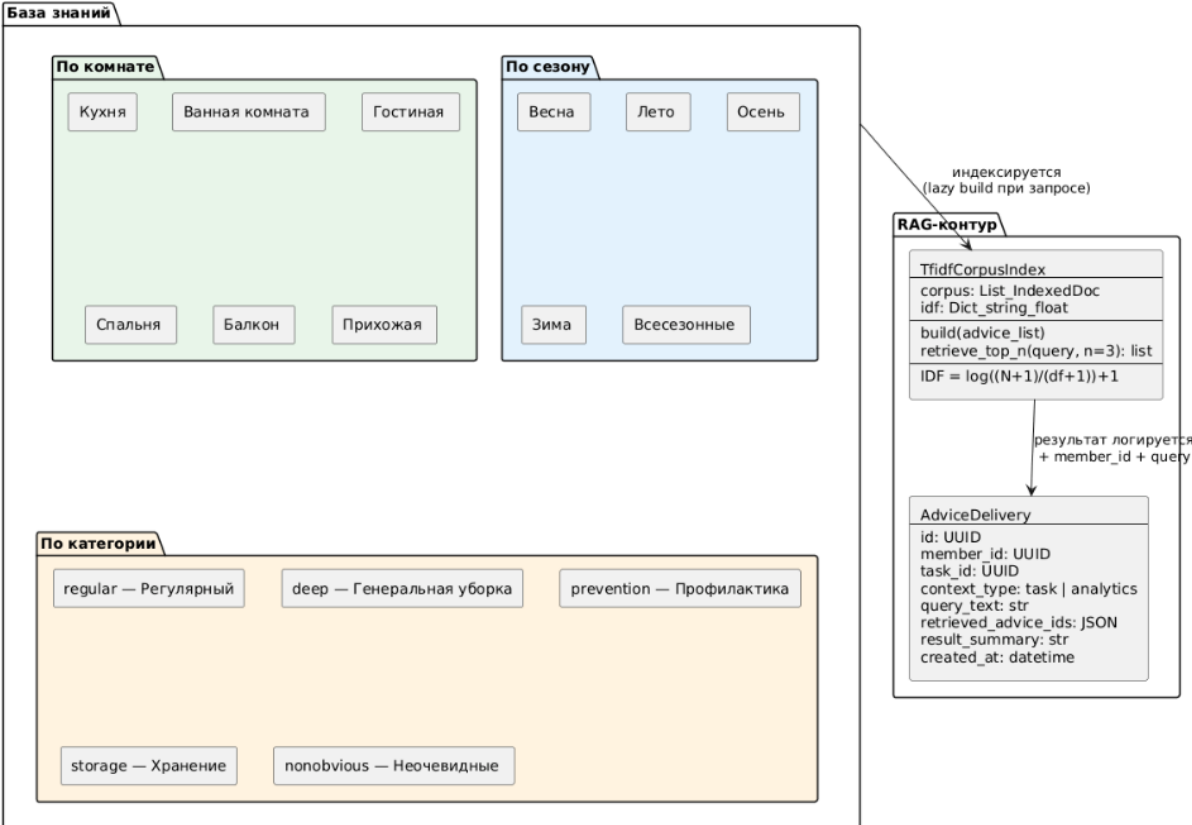


Рисунок 2 – Структура базы знаний

2.2 Алгоритм расчёта повторений и обработки пропусков

Повторяемые бытовые задачи требуют формализованного описания периодичности и единообразного механизма генерации экземпляров на календарном горизонте. В работе для описания повторений используется модель, совместимая с iCalendar, где повторяемость задаётся правилом RRULE и сопутствующими параметрами [24].

Ключевой проблемой предметной области является обработка пропусков: если задача не выполнена в плановый день, система должна определить дальнейшее поведение расписания. В работе применяются три

политики: перенос невыполненного экземпляра в ближайшее допустимое окно, фиксация просрочки без переноса и пропуск экземпляра без образования долга. Выбор подхода фиксируется на уровне шаблона задачи, что обеспечивает предсказуемость планирования и снижает отход от плана при сериях пропусков.

2.2.1 Входные и выходные данные

Вход алгоритма включает шаблон задачи с правилом повторения RRULE, параметрами окна выполнения и выбранной политикой пропусков, а также журнал событий, содержащий отметки выполнения, пропуска и переноса. Выходом является множество экземпляров задач на заданный период с вычисленными статусами и, при необходимости, сформированными событиями переноса.

Псевдокод алгоритма генерации экземпляров и статусов представлен на Листинге 2.

```
Алгоритм BuildOccurrences(Шаблон T, Журнал E, Дата from, Дата
to):
  D ← GenerateRRULE(T.RRULE, from, to) // генерация дат по RFC 5545
  OccList ← ∅

  для каждой даты d из D:
    occ ← (T.id, d)
    если Exists(E, occ, "done"):
      status ← DONE
    иначе если Exists(E, occ, "skipped"):
      status ← SKIPPED
    иначе если Now ≤ d + T.window:
      status ← ACTIVE
    иначе:
      если T.policy == OVERDUE:
        status ← OVERDUE
      иначе если T.policy == SKIP_NO_DEBT:
        status ← SKIPPED_AUTO
      иначе если T.policy == MOVE_FORWARD:
        d2 ← NearestAllowedDateAfter(Now, T)
        Append(E, Reschedule(occ, d2))
        status ← MOVED
```

```

OccList ← OccList U (occ, status)
вернуть OccList

```

Листинг 2 – Псевдокод алгоритма генерации экземпляров и статусов

Сложность алгоритма линейна по числу сгенерированных дат на заданном горизонте планирования при индексированном доступе к событиям по ключу экземпляра. При реализации генератора RRULE требуется учитывать правила корректности повторений, закреплённые в спецификации iCalendar, включая обработку исключений и границ периода [44].

Диаграмма архитектуры системы представлена в виде диаграммы контекста (C4 Level 1) на Рисунке 3.

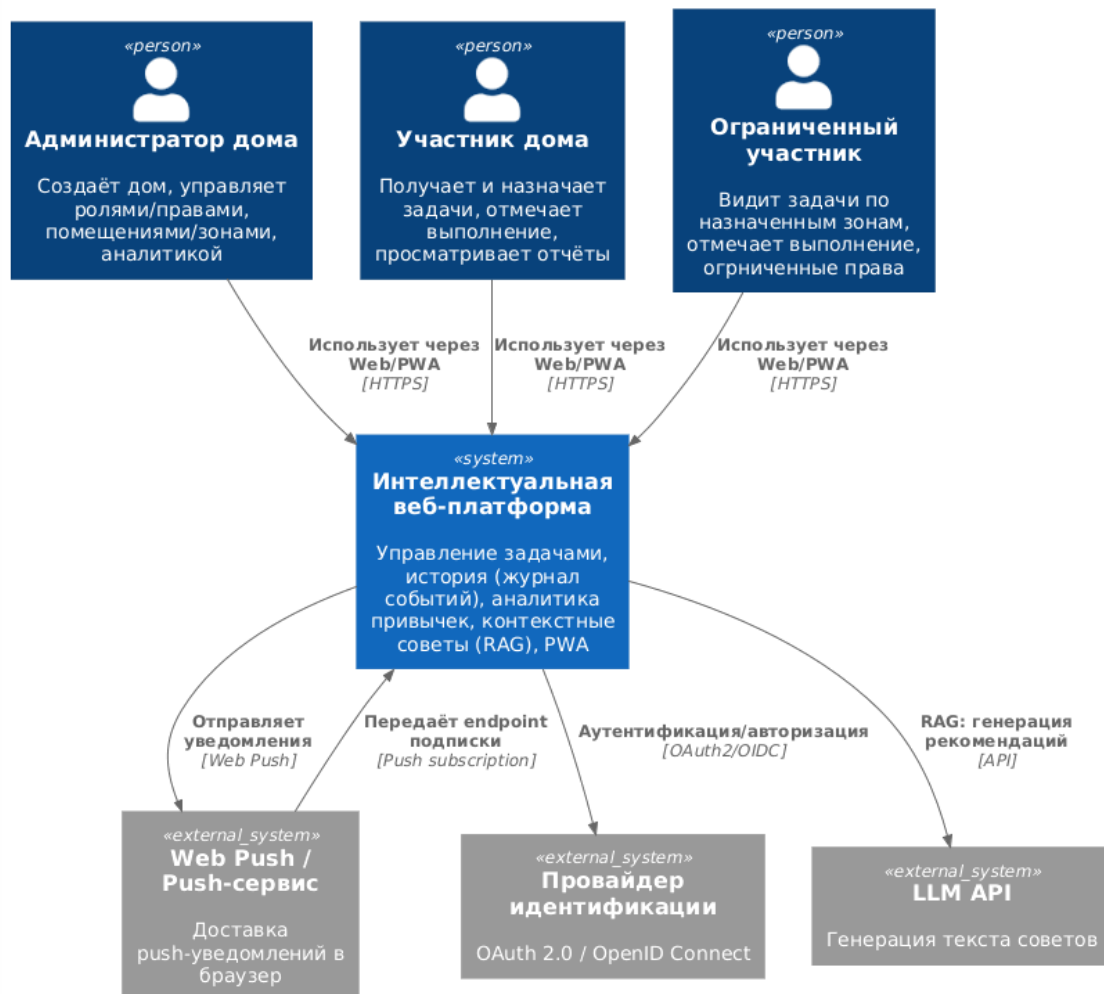


Рисунок 3 – Диаграмма контекста C4 Level 1

Диаграмма контекста отражает окружение разрабатываемой платформы и границы ответственности системы на уровне основных участников и внешних сервисов. Центральным элементом является интеллектуальная веб-платформа управления бытовыми задачами, доступ к которой осуществляется через веб-клиент с поддержкой PWA. Взаимодействие с системой выполняют три категории пользователей: администратор дома, участник дома и ограниченный участник, что согласуется с требованием разграничения доступа по ролям и контекстам.

На диаграмме также показаны внешние технические сервисы, необходимые для реализации требований уведомлений и безопасности. Механизм push-уведомлений реализуется через инфраструктуру Web Push и спецификацию Push API, где платформа формирует сообщения и инициирует их доставку на клиентские устройства. Сервис идентификации по протоколам OAuth 2.0 или OpenID Connect является опциональным компонентом и может использоваться для внешней аутентификации; при отсутствии внешнего провайдера применяется локальная аутентификация. Для модуля контекстных советов допускается использование внешнего компонента генерации текста, однако в базовом варианте рекомендации формируются в контуре платформы на основе извлечения данных из базы знаний и истории выполнения.

Процесс аналитики привычек и формирования рекомендаций представлен BPMN-диаграммой с разделением действий пользователя и серверной платформы. Пользователь запрашивает отчёт, задавая период и уровень агрегации, после чего платформа извлекает события из журнала, фильтрует их по дому и участнику и рассчитывает метрики соблюдения, серии, доли выполнений, просрочек и распределения нагрузки. Результаты возвращаются в виде таблиц и графиков.

После просмотра отчёта проверяется условие уровня соблюдения графика. Если отклонений нет, рутина сохраняется. При низком соблюдении

запускается подпроцесс рекомендаций: пользователь выбирает проблемную область, платформа формирует контекст, извлекает релевантные фрагменты из базы знаний и выдаёт рекомендации по корректировке рутины и контекстные советы. Факт выдачи фиксируется в журнале для трассируемости. Диаграмма отражает замкнутый контур: аналитика обеспечивает обратную связь, а рекомендации поддерживают управленческую коррекцию поведения на основе истории и базы знаний.

Процесс аналитики привычек и генерации рекомендаций представлен на Рисунке 4 в виде BPMN диаграммы.

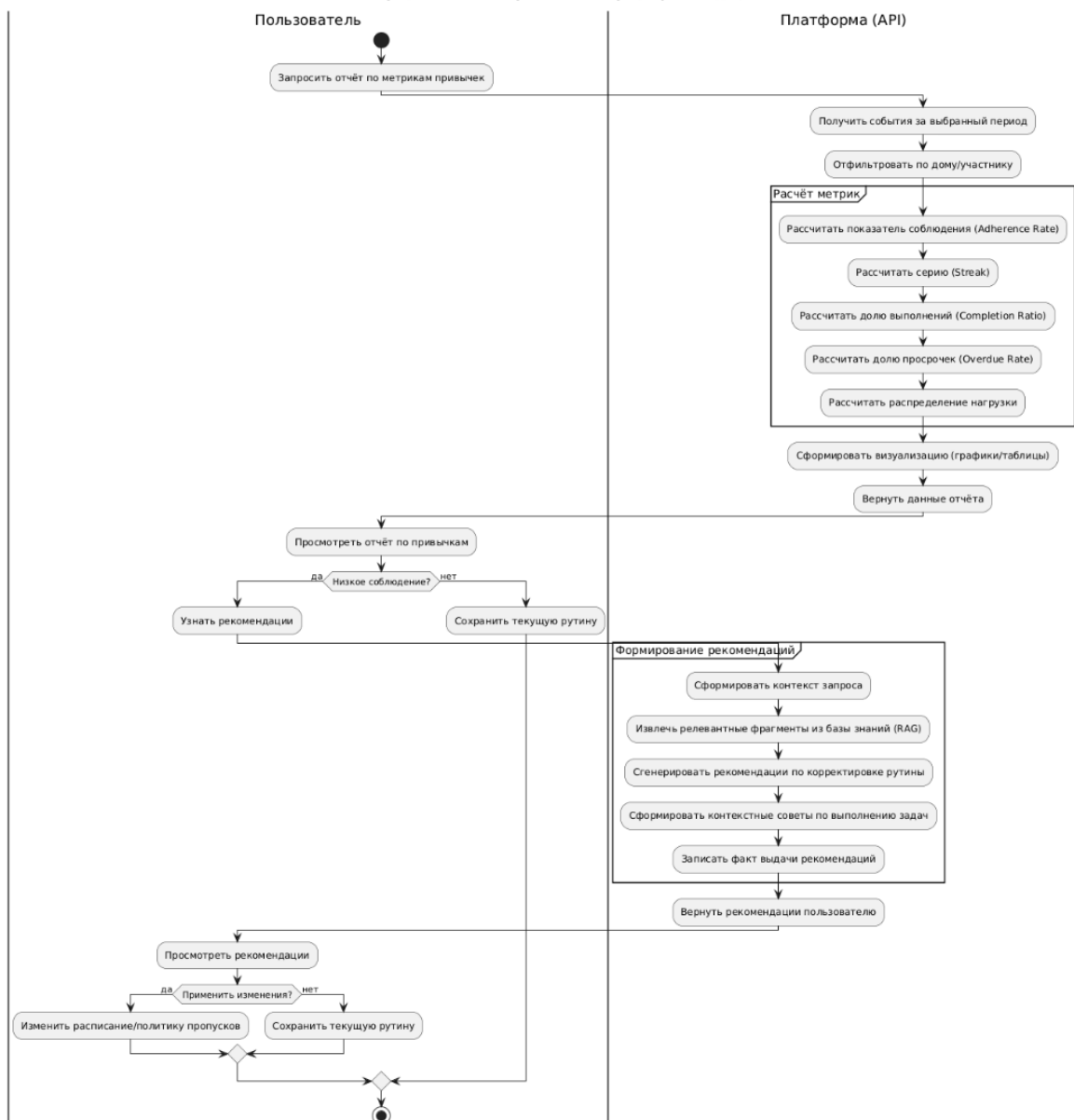


Рисунок 4 – Диаграмма процесса аналитики привычек и генерации рекомендаций

2.3 Проектирование пользовательского интерфейса

В рамках разработки интеллектуальной веб-платформы управления бытовыми задачами было выполнено проектирование ключевых экранов пользовательского интерфейса. Основной акцент сделан на принципах доступности, снижении когнитивной нагрузки и обеспечении кроссплатформенности в рамках концепции Progressive Web Apps (PWA).

2.3.1 Главный экран

Центральный интерфейс системы представляет собой персонализированную панель управления, обеспечивающую мгновенный доступ к наиболее значимой информации. В верхней части экрана располагается приветствие пользователя и визуальная шкала прогресса выполнения задач на текущий день. Основную область занимает список активных задач на текущий день с индикаторами приоритета, метками помещения и элементами управления для оперативной отметки статуса (выполнено, пропущено, перенесено). Визуальная шкала прогресса позволяет пользователю оценить текущий объем выполненных работ относительно запланированного. Помимо задач на текущий день при прокрутке экрана отображаются задачи на текущую неделю, месяц, а также задачи, просроченные в недавнее время.

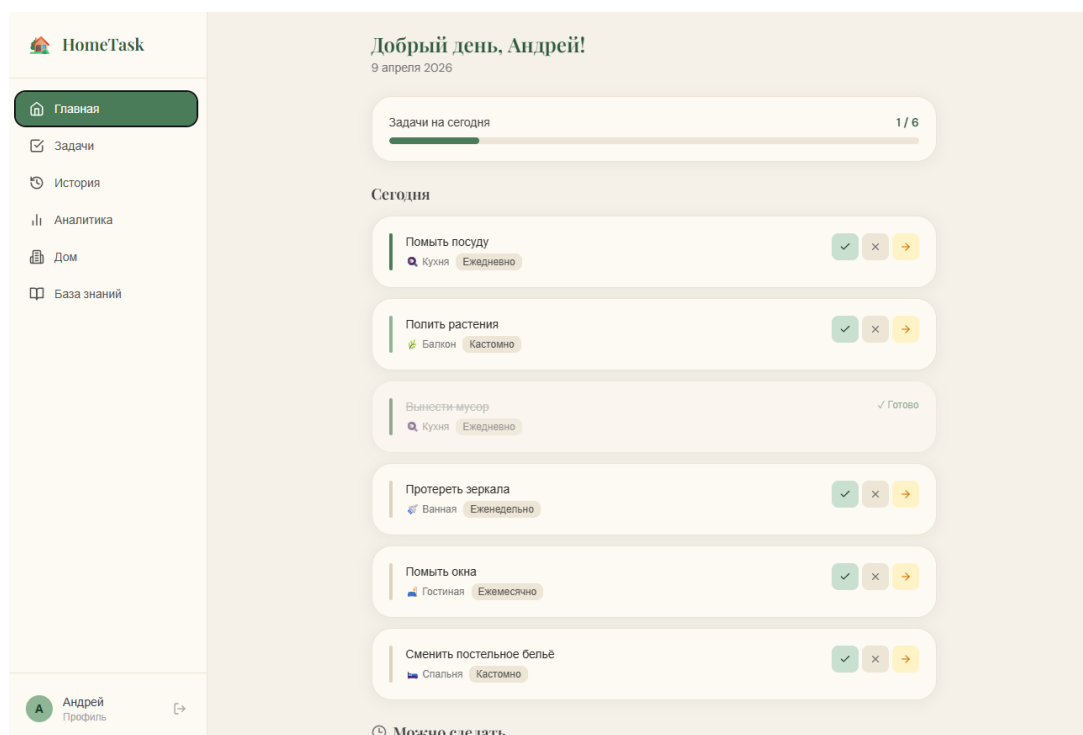


Рисунок 5 – Главный экран

2.3.2 Модуль создания и планирования задач

Модуль создания и планирования задач представляет собой специализированную форму ввода, предназначенную для детального задания параметров новой задачи. В верхней части окна располагаются элементы выбора помещения и наименования задачи, а также поле для ввода дополнительного описания. Ниже размещены средства настройки приоритета, позволяющие классифицировать задачу по степени важности, и блок выбора частоты выполнения.

Существенной частью модуля является настройка политики пропуска, определяющая поведение системы при невыполнении задачи в установленный срок. Дополнительно пользователь может указать дату начала выполнения и назначить ответственного исполнителя. Наличие шаблонов задач для выбранного помещения упрощает ввод типовых действий и сокращает время планирования. Таким образом, данный модуль обеспечивает гибкую настройку задач с учетом особенностей домашнего хозяйства, периодичности работ и распределения обязанностей между пользователями.

Новая задача

КОМНАТА

Ванная

Шаблоны задач для «Ванная»

НАЗВАНИЕ

Название задачи

ОПИСАНИЕ

Опционально

ПРИОРИТЕТ

Низкий Средний Высокий

ЧАСТОТА

Разово Ежедневно Еженедельно Ежемесячно

Кастомный

ПОЛИТИКА ПРОПУСКА

Перенести Просрочить Пропустить

ДАТА НАЧАЛА

09.04.2026

ОТВЕТСТВЕННЫЙ

— Не назначен —

Рисунок 6 – Модуль создания и планирования задач

2.3.3 Аналитическая панель привычек

Аналитическая панель привычек предназначена для отображения статистики выполнения домашних задач и оценки регулярности их выполнения. В верхней части экрана располагаются основные сводные показатели, отражающие уровень соблюдения плана, серию выполнений, долю завершенных и просроченных задач, а также показатель стабильности.

Основную область занимает блок анализа привычек с выводами по помещениям, задачам, пользователям и временным закономерностям. Цветовые акценты и графические элементы позволяют выявлять проблемные зоны, отслеживать тенденции и принимать решения по корректировке частоты задач и распределению нагрузки.

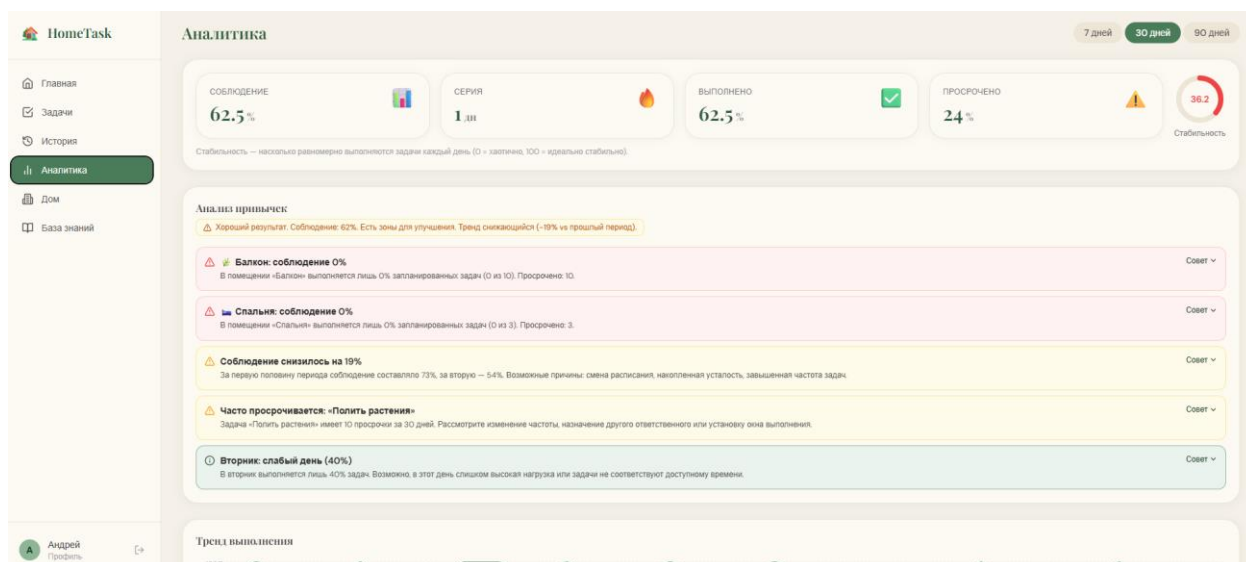


Рисунок 7 – Аналитическая панель привычек

2.3.4 Реестр задач

Реестр выполнения задач представляет собой экран учета, предназначенный для просмотра истории действий пользователей в системе. В верхней части страницы расположены фильтры по статусу, помещению и временному интервалу, что упрощает поиск и анализ записей за выбранный период.

Основную область интерфейса занимает хронологический список задач, для которых отображаются наименование, помещение, автор, плановая дата и итоговый статус. Для каждой записи также фиксируются дата и время изменения состояния. Данный экран используется для контроля выполнения задач и анализа пользовательской активности.

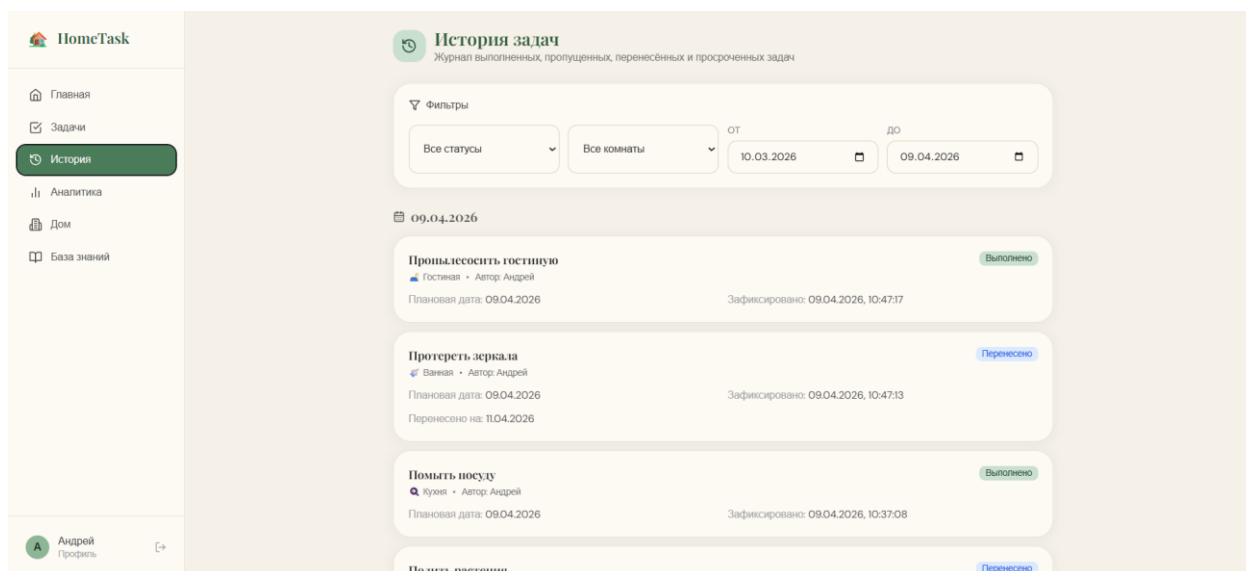


Рисунок 8 – Реестр задач

2.3.5 Мобильный интерфейс PWA

Адаптированная версия интерфейса спроектирована с учетом специфики мобильного взаимодействия. Крупные элементы управления обеспечивают удобство отметки выполнения задач «на ходу». Нижняя панель навигации предоставляет быстрый доступ к ключевым разделам системы, а механизмы Push-уведомлений гарантируют оперативное информирование пользователя о новых назначениях и приближающихся сроках выполнения критически важных задач.

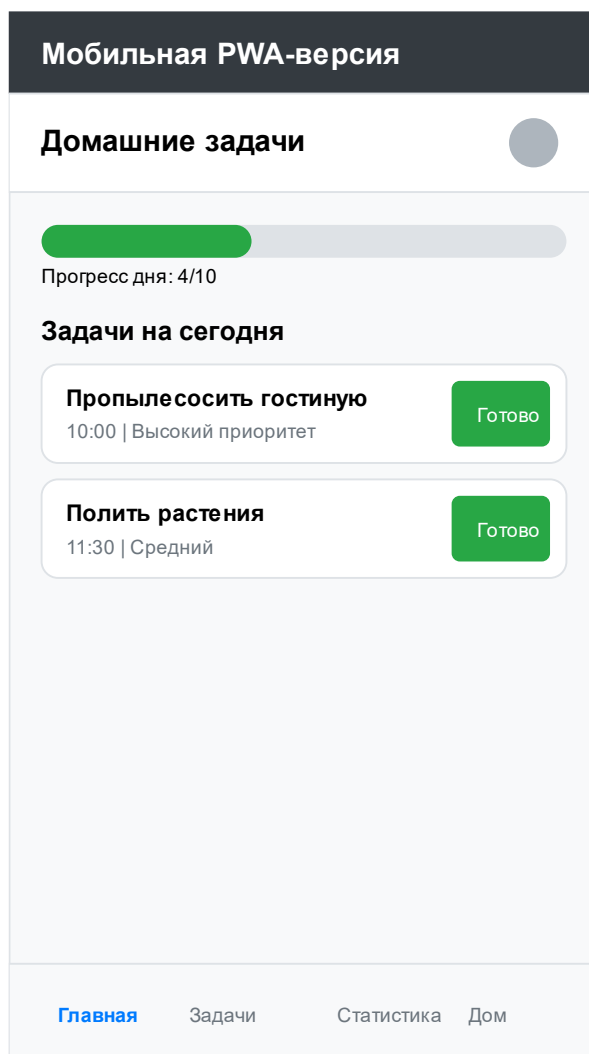


Рисунок 9 – Мобильный интерфейс PWA

2.4 Техническая реализация программного продукта

Техническая реализация интеллектуальной платформы выполнена в виде full-stack веб-приложения, включающего серверную часть, клиентскую часть, базу данных и инфраструктуру развёртывания. Выбранный подход соответствует выбранной архитектуре системы и требованиям к кроссплатформенности, модульности и поддержке аналитических функций. Основу решения составляют серверный API, реляционная база данных и веб-клиент с адаптацией под настольные и мобильные устройства.

2.4.1 Серверная часть

Серверная часть реализована на языке Python с использованием фреймворка FastAPI. Данный инструмент выбран из-за поддержки асинхронной обработки запросов, удобной валидации входных данных и возможности построения REST API. На сервере реализованы модули аутентификации, управления задачами, помещениями и участниками дома, фиксации событий выполнения, расчёта аналитики привычек и формирования контекстных советов.

Для описания структуры данных используются схемы Pydantic, а доступ к базе данных организован через SQLAlchemy. Подход с выделением слоёв API, бизнес-логики и моделей данных обеспечивает сопровождаемость и расширяемость программного продукта, что соответствует требованию о модульной организации кода.

2.4.2 База данных и хранение данных

В качестве системы управления базами данных используется PostgreSQL. База данных предназначена для хранения пользователей, домов, участников, помещений, задач, событий выполнения и элементов базы знаний. Такая модель позволяет фиксировать как текущее состояние системы, так и историю действий пользователей, необходимую для расчёта аналитических показателей и формирования рекомендаций. Выбор PostgreSQL согласуется с требованиями к реляционной модели данных, миграциям схемы и аналитической обработке истории выполнения.

Для управления изменениями структуры базы данных используются миграции Alembic. Это обеспечивает воспроизводимость развёртывания и единообразное обновление схемы на различных средах.

2.4.3 Клиентская часть

Клиентская часть реализована с использованием React и TypeScript, что обеспечивает компонентный подход к разработке интерфейса, типизацию данных и удобство сопровождения проекта. Взаимодействие с серверным API выполняется через HTTP-запросы, а пользовательский интерфейс включает основные экраны системы: главную страницу, модуль задач, аналитическую панель, раздел дома, историю выполнения и базу знаний.

Для отображения аналитических данных используются графические компоненты и диаграммы, а интерфейс адаптирован для работы как на настольных устройствах, так и на мобильных экранах. Это соответствует требованиям к веб-платформе с PWA.

2.4.4 Состав программного кода и используемые средства

Программный продукт включает несколько групп файлов: серверный код на Python, клиентский код на TypeScript, конфигурационные файлы окружения, файлы миграций базы данных и инфраструктурные файлы контейнерного развёртывания. Таким образом, в состав решения входят не только прикладные модули, но и средства конфигурирования, сборки и запуска системы.

В числе основных используемых библиотек и инструментов можно выделить FastAPI, SQLAlchemy, Pydantic, Alembic, PostgreSQL, React и TypeScript. Для развёртывания применяется Docker, что позволяет запускать систему в воспроизводимом окружении.

2.4.5 Требования к программному и техническому обеспечению

Для функционирования системы требуется серверное окружение с поддержкой Python, PostgreSQL и контейнеризации Docker либо эквивалентная локальная установка компонентов. Клиентская часть работает

в современном браузере на настольном компьютере, планшете или смартфоне. Минимальные требования к стенду включают 2 vCPU, 4 ГБ оперативной памяти и SSD-хранилище, что соответствует условиям эксплуатации. Передача данных между клиентом и сервером должна выполняться по HTTPS, а данные аутентификации должны храниться в защищённом виде.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Daminger A. The Cognitive Dimension of Household Labor. American Sociological Review. 2019. URL: <https://journals.sagepub.com/doi/10.1177/0003122419859007> (дата обращения: 25.02.2026).
- [2] ВЦИОМ. Идеальная семья – 2024. Аналитический обзор. 08.07.2024. URL: <https://wciom.ru/analytical-reviews/analiticheskii-obzor/idealnaja-semja-2024> (дата обращения: 25.02.2026).
- [3] Росстат. Как расходую время мужчины и женщины (по итогам выборочного наблюдения использования суточного фонда времени населением). URL: <https://11.rosstat.gov.ru/storage/mediabank/Как%20расходую%20время%20мужчины%20и%20женщины.pdf> (дата обращения: 25.02.2026).
- [4] ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы. URL: <https://docs.cntd.ru/document/1200006924> (дата обращения: 25.02.2026).
- [5] ГОСТ Р ИСО/МЭК 25010-2015. Модели качества систем и программных продуктов. URL: <https://protect.gost.ru/document.aspx?control=7&id=200427> (дата обращения: 25.02.2026).
- [6] ISO. ISO/IEC 25010 (описание стандарта). URL: <https://www.iso.org/standard/35733.html> (дата обращения: 27.02.2026).
- [7] ФГБУ «Институт стандартизации». ISO/IEC/IEEE 29148:2018 (страница каталога). URL: <https://www.gostinfo.ru/catalog/Details/?id=6439527> (дата обращения: 25.02.2026).
- [8] ISO/IEC/IEEE 29148:2018 (описание в ISO ОВР). URL: <https://www.iso.org/obp/ui/en/> (дата обращения: 25.02.2026).

- [9] NIST. SP 800-162: Guide to Attribute Based Access Control (ABAC). URL: <https://csrc.nist.gov/publications/detail/sp/800-162/final> (дата обращения: 25.02.2026).
- [10] OWASP. OWASP Top 10 (2021). URL: <https://owasp.org/Top10/2021/> (дата обращения: 25.02.2026).
- [11] Chen P. P. The Entity-Relationship Model—Toward a Unified View of Data. ACM TODS. 1976. URL: <https://dl.acm.org/doi/10.1145/320434.320440> (дата обращения: 25.02.2026).
- [12] Wixted J. T. The psychology and neuroscience of forgetting. Annual Review of Psychology. 2004. URL: <https://pubmed.ncbi.nlm.nih.gov/14744216/> (дата обращения: 25.02.2026).
- [13] Григорьева В. Н. Проспективная память: обзор. 2022. URL: <https://cyberleninka.ru/article/n/prospektivnaya-pamyat-obzor> (дата обращения: 25.02.2026).
- [14] Lally P. et al. How are habits formed. Eur. J. Social Psychology. 2010. URL: <https://onlinelibrary.wiley.com/doi/10.1002/ejsp.674> (дата обращения: 25.02.2026).
- [15] Fogg B. J. A behavior model for persuasive design. ACM. 2009. URL: <https://dl.acm.org/doi/10.1145/1541948.1541999> (дата обращения: 25.02.2026).
- [16] Sweller J. Cognitive load during problem solving: Effects on learning. Cognitive Science. 1988. URL: <https://www.sciencedirect.com/science/article/pii/0364021388900237> (дата обращения: 25.02.2026).
- [17] Haynes A. B. et al. A Surgical Safety Checklist... NEJM. 2009. URL: <https://www.nejm.org/doi/full/10.1056/NEJMsa0810119> (дата обращения: 25.02.2026).
- [18] Вестник Росздравнадзора (Mednet). Применение чек-листов: обзор/практика. URL: <https://vestnik.mednet.ru/content/view/1768/30/lang/ru/> (дата обращения: 25.02.2026).

- [19] MDN (RU). Прогрессивные веб-приложения. URL: https://developer.mozilla.org/ru/docs/Web/Progressive_web_apps (дата обращения: 25.02.2026).
- [20] MDN (RU). Манифест веб-приложения. URL: https://developer.mozilla.org/ru/docs/Web/Progressive_web_apps/Manifest (дата обращения: 25.02.2026).
- [21] W3C. Web Application Manifest. URL: <https://www.w3.org/TR/appmanifest/> (дата обращения: 27.02.2026).
- [22] W3C. Service Workers. URL: <https://www.w3.org/TR/service-workers/> (дата обращения: 27.02.2026).
- [23] W3C. Push API. URL: <https://www.w3.org/TR/push-api/> (дата обращения: 27.02.2026).
- [24] IETF. RFC 5545 iCalendar. URL: <https://datatracker.ietf.org/doc/html/rfc5545> (дата обращения: 25.02.2026).
- [25] IETF. RFC 8030 Web Push Protocol. URL: <https://datatracker.ietf.org/doc/html/rfc8030> (дата обращения: 25.02.2026).
- [26] web.dev. Push notifications overview. URL: <https://web.dev/articles/push-notifications-overview> (дата обращения: 25.02.2026).
- [27] PostgreSQL Docs. JSON Types. URL: <https://www.postgresql.org/docs/current/datatype-json.html> (дата обращения: 25.02.2026).
- [28] PostgreSQL Docs. Table Partitioning. URL: <https://www.postgresql.org/docs/current/ddl-partitioning.html> (дата обращения: 25.02.2026).
- [29] PostgreSQL Docs. Release Notes. URL: <https://www.postgresql.org/docs/release/> (дата обращения: 25.02.2026).
- [30] FastAPI. Официальная документация. URL: <https://fastapi.tiangolo.com/> (дата обращения: 25.02.2026).
- [31] Flask. Официальная документация. URL: <https://flask.palletsprojects.com/> (дата обращения: 25.02.2026).

- [32] Django Project. Официальный сайт. URL: <https://www.djangoproject.com/> (дата обращения: 25.02.2026).
- [33] Google Calendar Help. Create & manage tasks. URL: <https://support.google.com/calendar/answer/9901136> (дата обращения: 25.02.2026).
- [34] Microsoft Support. Introduction to the Outlook Calendar. URL: <https://support.microsoft.com/en-us/office/introduction-to-the-outlook-calendar-d94c5203-77c7-48ec-90a5-2e2bc10bd6f8> (дата обращения: 25.02.2026).
- [35] FamilyWall. Официальный сайт. URL: <https://www.familywall.com/en/index.html> (дата обращения: 25.02.2026).
- [36] FamilyWall. Google Play listing. URL: <https://play.google.com/store/apps/details?hl=en&id=com.familywall> (дата обращения: 25.02.2026).
- [37] Nipto. Google Play listing. URL: <https://play.google.com/store/apps/details?hl=en&id=com.nipto.niptoapp> (дата обращения: 25.02.2026).
- [38] Tody. Google Play listing. URL: <https://play.google.com/store/apps/details?hl=en&id=com.looploop.tody> (дата обращения: 25.02.2026).
- [39] Flatastic. Официальный сайт. URL: <https://www.flatastic-app.com/> (дата обращения: 25.02.2026).
- [40] Flatastic. Google Play listing. URL: <https://play.google.com/store/apps/details?hl=en&id=com.flatastic.app> (дата обращения: 25.02.2026).
- [41] Lewis P., Perez E., Piktus A. и др. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. URL: <https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf> (дата обращения: 27.02.2026).

[42] Karpukhin V., Oguz B., Min S. и др. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020. URL: <https://aclanthology.org/2020.emnlp-main.550/> (дата обращения: 27.02.2026).

[43] Faiss. A library for efficient similarity search and clustering of dense vectors. URL: <https://github.com/facebookresearch/faiss> (дата обращения: 27.02.2026).

[44] IETF. RFC 5545: Internet Calendaring and Scheduling Core Object Specification (iCalendar). URL: <https://datatracker.ietf.org/doc/html/rfc5545> (дата обращения: 27.02.2026).

[45] Яндекс Cloud. RAG: учим искусственный интеллект работать с новыми данными. URL: <https://yandex.cloud/ru/blog/posts/2025/05/retrieval-augmented-generation-basics> (дата обращения: 27.02.2026).

[46] MDN Web Docs (RU). Прогрессивные веб-приложения. URL: https://developer.mozilla.org/ru/docs/Web/Progressive_web_apps (дата обращения: 27.02.2026).

[47] NASA. Technical Memorandum NASA-TM-2010-216396 (PDF). URL: <https://hsi.arc.nasa.gov/flightcognition/Publications/NASA-TM-2010-216396.pdf> (дата обращения: 25.02.2026).

[48] Sweller, J. CHAPTER TWO — Cognitive Load Theory. In: *Psychology of Learning and Motivation*. Vol. 55. 2011. P. 37–76. URL: <https://www.sciencedirect.com/science/chapter/bookseries/abs/pii/B9780123876911000028> (дата обращения: 25.02.2026).

[49] Fogg, B. J. Captology: Fogg Behavior Model (PDF). URL: https://www.demenzemedicinagenerale.net/images/menssana/Captology_Fogg_Behavior_Model.pdf (дата обращения: 25.02.2026).

[50] PubMed Central (PMC). Статья в открытом доступе (идентификатор: PMC3505409). URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC3505409/> (дата обращения: 25.02.2026).

[51] PubMed Central (PMC). Статья в открытом доступе (идентификатор: PMC6086200). URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6086200/> (дата обращения: 25.02.2026).